ClickFix First Aid — Architectural & Risk Analysis

Prepared by: Lead Software Architect review
Scope of evidence: `ClickFix-bypass-harness.zip` (full source, 40 files) and `ClickFix-First-Aid-v0.1.0-security-review.pdf` (16 pages).
Important scoping note: the uploaded zip contains only the *bypass harness* (harmless PoC pages + Playwright runner), **not** the extension source itself. The application architecture below is reconstructed from the review PDF's scope section (which enumerates `extension/`, `engine/`, `rules/`, `checker/`, `guide/` files with specific line references). Conclusions about the harness come from static inspection of the actual code; conclusions about the extension are cross-checked against the review's live-verified findings and are labelled accordingly.

1. System Overview

ClickFix First Aid is a defense-in-depth tool against "ClickFix" social-engineering attacks (fake CAPTCHA pages that trick users into pasting malicious commands into the Windows Run dialog). It has five cooperating subsystems:

Subsystem	Files (per review scope)	Responsibility
Clipboard interception	`extension/page-hook.js`, `extension/main-world.js`	MAIN-world wrappers around `Clipboard.writeText`, `Clipboard.write`, `DataTransfer.setData`, `execCommand('copy')`
Page analysis & UI	`extension/content.js`, `extension/ui.js`	Isolated-world listener, page-text collection, verdict-driven UI (custom element `<clickfix-first-aid-ui data-state>`)
Detection engine	`engine/detector.js`	Pure decision function `decide()` + `normalize()`
Rules (data-driven)	`rules/clickfix-rules.json`	Declarative signal groups: executors, remote, stealth, decoy, lure themes
Host-side recovery	`checker/src/*.ps1` (Check-ClickFix.ps1, Protect-RunDialog.ps1), `guide/recovery.html`	Windows registry/event-log scanning, Run-dialog hardening, victim recovery guide

Harness (actual code inspected): `run.mjs` — a Node HTTP server serving PoC pages on an ephemeral port, driving Playwright Chromium with the unpacked extension loaded (`--load-extension`), granting clipboard permissions, clicking each page's button, and reading back (a) the real clipboard text and (b) the extension UI's `data-state`. The 40 PoC pages each encode one attack class (realm borrowing, TOCTOU, prototype poisoning, late lure insertion, CSS/shadow content, forged events, false-positive tutorials).

Architectural pattern

A classic **pipeline with a two-world trust boundary**:

```
[Attacker page]                     [Isolated world]                [Host]
   writeText(x) ---CustomEvent--->  content.js listener
        |                                |
   evaluate(text)                    analyzeThisPage()  (innerText + textContent)
        |                                |
        |                            detector.decide()  <-- rules/clickfix-rules.jso
        |                                |
        |<-- verdict ------------------ UI (ui.js, shadow DOM)
        |
   block? apply(native, [SAFE_TEXT]) : apply(native, [x])
```

The design is a **policy/mechanism split** (engine = policy, page-hook = mechanism) with rules externalized to JSON — a sound pattern. The critical weakness is that the channel between the two worlds is an **unauthenticated, page-observable DOM CustomEvent**, and the final enforcement (replacing the argument before the native call) happens back in the page-controlled MAIN world.

2. Structural Dependencies

1. **rules/clickfix-rules.json → detector.js** — the engine is data-driven; a rules regression silently changes every verdict. Currently 13 of 14 of the review's block fixtures fail (review §"New test cases", p.15).
2. **page-hook.js → detector.js → content.js → ui.js** — a linear chain; any broken link fails *open* (payload reaches clipboard) rather than closed.
3. **MAIN world ↔ isolated world channel** — the CustomEvent `clickfix-first-aid` on `document`. Both directions are subvertible: forged events *in* (finding C1), `stopImmediatePropagation`/flood *out* (finding A10).
4. **content.js ↔ background service worker ↔ top frame** — subframe reports are forwarded to the top frame through the background worker; cross-origin frames can therefore inject UI onto the parent site (C1, `forged_frame.html`).
5. **ui.js → content.js state flags** — `flagged = true` is set before `showLureBanner()` mounts (C2), a hidden temporal coupling.
6. **checker scripts → WScript.Shell COM, Get-WinEvent, registry** — the Startup check dies entirely if WSH is disabled (D5); MSI parsing depends on English-only message strings (D2).
7. **run.mjs → Playwright + Chromium + the extension at a hard-coded path** (`EXT` constant) — the harness itself has a hard dependency on `/home/user/workspace/cf/clickfix-first-aid/extension`, so it is not portable without editing.

3. Data Models

Core domain objects (derived from review findings and harness behavior):

- **Command text** (`text`) — the untrusted payload string. Flows through `evaluate()` in page-hook, then `decide()` in the engine.
- **Verdict** — enum `allow | warn | block` (the review proposes adding `hold`). This is the system's single most important data type: **only** `block` **mutates the clipboard** (replaced with `SAFE_TEXT`); `warn` leaves the payload intact (finding A1, Critical).
- **Signals** — `executors[]`, `remotes[]`, `stealthScore` (ps-encoded, hidden-window rules), `decoy` (fake-verification-comment rule), `page.weak`/`page.strong` (lure context), `subframe`, line count (`maxLinesForAutoBlock = 5`).
- **SAFE_TEXT** — the replacement string written on block. Its comment claims shells treat # as a comment, which is wrong for interactive macOS zsh (C4).
- **Settings object** — `chrome.storage` object holding soft-warnings toggle and muted sites; loaded once at page start and written back wholesale (C3 — stale read-modify-write).
- **UI state** — `data-state` attribute on the `<clickfix-first-aid-ui>` custom element (`blocked / warned / lure / null`), rendered inside a closed shadow root.
- **Host-side data** — `HKCU\...\Explorer` Run keys, `DisabledHotkeys` value, MSI event 1033 records, AppData file-scan results.

4. Data Lifecycle (clipboard payload)

1. **Creation** — attacker page composes the payload (or forges a report event entirely, C1).
2. **Capture** — user click invokes a wrapped clipboard API. `evaluate(text)` stringifies the argument. **Page-controlled objects get stringified twice** — once by the hook, once by the native call — enabling the TOCTOU swap (A2, `tostring.html`).

```
1. **Analysis** — `content.js`/`analyzeThisPage()` synchronously reads `innerText` (
```

1. **Decision** — `decide()` computes the verdict. **Fail-open hazard:** if `Array.prototype[0]` is an accessor, every match list stays empty and the verdict is `allow` (A4, `arrayproto.html`); `Object.prototype.lastIndex` poisoning does the same to the regexes (`regexproto.html`).
2. **Enforcement** — on `block`, the hook replaces the argument with `SAFE_TEXT` before the native call. On `warn`, nothing is replaced (A1 — the Critical finding).
3. **Notification** — CustomEvent dispatched on `document`; content.js throttles all reports globally at 800 ms (A10 — floodable with one fake event, `flood.html`).
4. **Presentation** — toast (15 s), lure banner, or full-screen overlay via `ui.js`; the overlay renders page-supplied text (C1 — tech-support-scam vector).
5. **Consumption** — the user pastes into the Run dialog (the actual harm point; out of the extension's reach).

6. **Recovery** — checker scripts scan Run keys/MSI events/new files; `guide/recovery.html` walks the victim through account revocation.

Lifecycle gap: steps 3–4 read attacker-controlled DOM state at a single instant; steps 5–7 run on an attacker-observable channel. Every lifecycle stage from 2 to 7 is influenced by the adversary.

5. Critical Failure Paths

A browser extension rarely "crashes" — the dominant failure mode is **silently failing open** (security control neutralized with no error). Ranked by severity:

1. **Engine core-dump into `allow` (A4/A11-class,** `arrayproto.html`, `regexproto.html`**)** — prototype poisoning empties all match lists; the detector effectively crashes while still returning a verdict. Live-verified payload-on-clipboard.
2. `warn` **= no protection (A1)** — off-lure (or lured-blind) block-level commands only warn; the payload stays on the clipboard. This is the system's central design flaw: the decision that a write is dangerous and the mutation of that write are coupled to page-context detection that is trivially blinded (shadow roots, CSS `::before`, cross-origin frames, late insertion, split junk text — 8 enumerated blinding methods in the review).
3. **Channel severance (A10)** — `stopImmediatePropagation` or a one-event flood silences all UI. Combined with A1, the user gets zero signal while the payload lands.
4. **Forged verdict injection (C1)** — any page or ad frame dispatches the CustomEvent and paints the trusted full-screen overlay with attacker text (`forged.html`, `forged_frame.html`).
5. **TOCTOU double-stringification (A2)** — `toString()` returns benign text on first call, payload on second; check and use diverge.
6. **Unhooked write paths (A3)** — `clipboardData.items.add()` and `clearData()` bypass the `setData` hook entirely (`itemsadd.html`).
7. **Checker collapse (D2/D5)** — WSH disabled loses the whole Startup section; non-English MSI parsing misreports every install. Host-side, not fail-open, but the recovery story breaks.

True "crash" candidates: B7 — the `write` wrapper awaiting `getType()` can push the native write past user-activation expiry, throwing `NotAllowedError` and breaking legitimate apps (fail-closed for the page, breakage for the user). D6 — `Stop-Process -Name explorer -Force` on the Windows side kills every Explorer window and races the shell restart.

6. Concurrency & Race Conditions

1. **TOCTOU on the payload (A2)** — the same untrusted value is read twice (hook `evaluate`, then native conversion); the value changes between reads.
2. **Point-in-time page analysis vs. late DOM mutation (**`late_instr.html`, `late_instr2.html`**)** — ordering race between the copy and the attacker's post-copy DOM insert; no re-scan window exists.
3. **Global 800 ms throttle (A10)** — a shared mutable timestamp; a single fake event consumes the window for a real block report; `allow` reports reset the timer. Cross-frame reports

interleave through the same throttle.

4. **Toast/banner ordering (C2)** — `flagged = true` is committed before the banner mounts; a warn that arrives first permanently suppresses the later lure banner (an interleaving of two UI actions with no rollback).
5. **Stale settings read-modify-write (C3)** — settings are read at page start and written back wholesale; two tabs (or a popup toggle racing a "Do not show on this site" click) lose updates — a classic lost-update race.
6. **MAIN/isolated world startup ordering** — the MAIN script must run before page scripts and the isolated-world listener must be in place before the first dispatch; the review explicitly flags this ordering as untested.
7. **Harness (`run.mjs`)** — single-threaded sequential page loop, ephemeral port; no concurrency issues of its own, but it depends on Playwright's `waitForEvent('serviceworker')` timing.

7. Top 5 Highest-Risk Components Requiring Rigorous Unit Testing

Ranked by impact × reachability (all five are on the attack path; failure of any is fail-open or trust-breaking).

1. `engine/detector.js` — `decide()` + `normalize()` (the decision core)

- **Why:** every verdict flows through it; its rules drive 13/14 of the review's block fixtures failing today. Prototype poisoning turns it into a silent `allow` machine.
- **Unit tests required:**
  - Run the engine with poisoned `Array.prototype['0']`, `Object.prototype.lastIndex`, and getters on `Object.prototype` — expect unchanged verdicts (regression for A4).
  - Unicode normalization: en/em dashes (U+2013–2015, U+2212), smart quotes, `/`-prefixed and abbreviated PowerShell parameters (`-wi hidden`, `-en`, `/enc`).
  - Executor-equivalents: `%ComSpec%`, `%windir:~-7,1%`, 8.3 short names, curl-to-Startup, UNC `@SSL/DavWWWRoot`, format-operator invocation.
  - Line counting: only non-empty, non-comment lines; decoy/stealth must not be cancelled by junk lines.
  - The full fixture table from the review (p.15) as a golden regression suite.
- **Boundary needing integration tests:** verdicts combined with real page contexts (`page.weak` vs `page.strong`, subframe).

2. `extension/page-hook.js` — clipboard API wrappers (the interception layer)

- **Why:** the only component that mutates the clipboard. TOCTOU (A2), unhooked paths (A3), realm borrowing, and forged events all land here.
- **Unit tests required:**

- **String-once semantics:** a `toString()` counter object must observe exactly one call; verdict must be computed on the same string the native call writes (regression for A2, `tostring.html`).
- Hook coverage matrix: `writeText`, `write` (multi-item, html-only `ClipboardItem`, promise-valued `ClipboardItem`), `setData`, `items.add`, `clearData`, `execCommand('copy')`.
- Realm isolation: fresh `about:blank` iframe (sync and late), `window.open('')`, cross-realm `DataTransfer.setData`, closed-shadow-root textarea + `execCommand`.
- Deferred-promise `ClipboardItem`: native write must not be delayed past user activation (B7).

3. The MAIN ↔ isolated world channel (`page-hook.js notify()` + `content.js` listener)

- **Why:** it is the trust root, and it is an unauthenticated DOM event. Forged overlays (C1) and UI silencing (A10) both exploit it; a hardening to `MessageChannel` is proposed in the review.
- **Unit tests required:**
  - Ordering: isolated-world listener registered before any MAIN-world dispatch (startup race).
  - Forged/flooded events rejected or rate-limited per verdict, never globally, and never reset by `allow` reports (`flood.html`, `stopprop.html`).
  - Cross-origin frame reports must not render top-frame UI without recomputation (`forged_frame.html`).
- **Boundary needing integration tests:** this component cannot be validated by unit tests alone — its correctness is defined by browser world-isolation semantics.

4. `extension/content.js` — `analyzeThisPage()` and page-context collection

- **Why:** the lure-page signal that gates auto-block. Both false negatives (8 blinding methods) and false positives (B1: real tutorials get full-screen blocks) stem from here.
- **Unit tests required (fixture DOMs):**
  - Text extraction: skip `script/style/template/noscript/textarea` (B2, `fp_nextdata.html`); walk open shadow roots; read `::before` content (`css_content*.html`); handle block-split text with hidden filler (`split_junk.html`); cross-origin frame steps (`xorigin_steps.html`); late-inserted instructions (`late_instr*.html`).
  - `weak` vs `strong` classification: "Press Win+R, paste" tutorials must not auto-block (`fp_tutorial.html`); `weak` should be warn/hold, not block.
  - Performance: analysis must not synchronously force layout on huge DOMs (B6).

5. `extension/ui.js` + `content.js` state machine (verdict presentation & settings)

- **Why:** the user-facing decision surface and its state flags have ordering bugs (C2, C3) that permanently suppress warnings.
- **Unit tests required:**
  - Verdict→state transitions: every legal event order (warn then lure, lure then warn, repeated blocks) must end in the correct final state; the banner must supersede the toast.
  - Concurrent settings: simulate two tabs doing read-modify-write; no lost updates (regression for C3).
  - Overlay content: page-supplied text must be sanitized or quarantined (C1 — no phone numbers/URLs in trusted chrome).

Runner-up (host side): `checker/src/*.ps1` — D1 (Run-key `PS*` filter skips real values), D2 (localized MSI messages), D3 (Turkish-I culture folding), D6 (Explorer kill). Needs Windows device validation; the review could only verify these by code reading.

8. Harness Quality Notes (actual code inspected)

- `run.mjs` correctly isolates state per page (fresh page, clipboard primed to `before`, UI state read from the custom element), and the payloads are genuinely harmless (`echo test`, `.example.invalid`, `203.0.113.x`) — safe to run anywhere.
- Hard-coded `EXT` path and `/tmp/bp` server root are the only portability defects; the `PORT` template substitution in served files is a neat touch but means harness pages are non-static.
- The harness doubles as the review's evidence base and the recommended e2e suite (review p.16: 10 bypass pages should end with `SAFE_TEXT`/hold; 4 false-positive pages should show no block).

9. Recommended Test Priority (Summary)

Priority	Component	Highest-value test
P0	`detector.js`	Poisoned-prototype invariance + golden fixture table
P0	`page-hook.js` wrappers	String-once TOCTOU regression
P1	World channel	Authenticated channel; per-verdict throttle
P1	`analyzeThisPage()`	Fixture-DOM extraction (shadow, CSS, frames) + FP suite
P2	UI state machine	Toast/banner ordering; concurrent settings merge
P3	PowerShell checkers	Localized-message and culture-invariance tests (needs Windows)

Method note: all statements about the harness are from static inspection of the uploaded source. All statements about the extension are cross-referenced to the review PDF's findings (A1–E10); items the review marks "verified (code)" or "unverified" are inherited here with the same confidence labels.